

## CPU

- Hardware is the physical component of a computer.
- Tech continues to evolve rapidly by:
  - Increased capacity, performance and reduced cost.

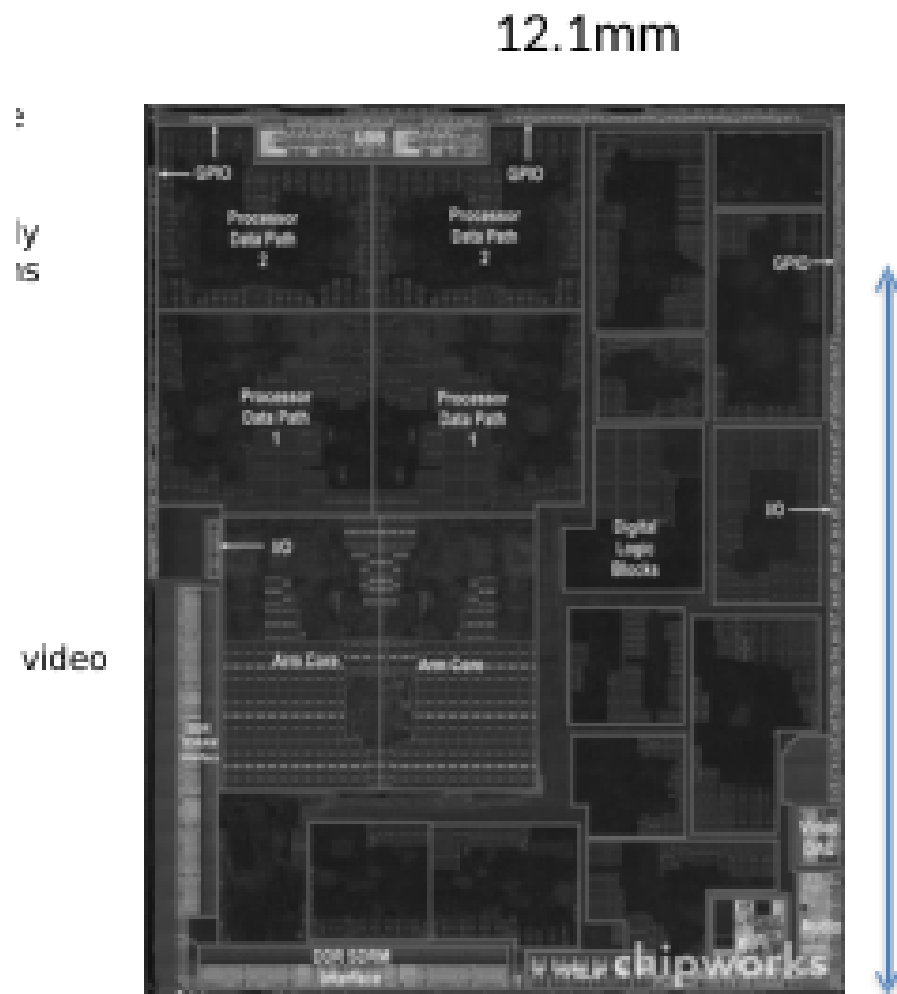
| Year | Technology                 | Relative performance/cost |
|------|----------------------------|---------------------------|
| 1951 | Vacuum tube                | 1                         |
| 1965 | Transistor                 | 35                        |
| 1975 | Integrated circuit (IC)    | 900                       |
| 1995 | Very large scale IC (VLSI) | 2,400,000                 |
| 2013 | Ultra large scale IC       | 250,000,000,000           |

Figure 1: Key Takeaway: performance has increased exponentially while cost has decreased

## Inside the PC

- Computer systems have four parts:
  1. Hardware: physical components (e.g., CPU, RAM, Disk, Mouse, Keyboard)
  2. Software: system software, applications, middleware
    - Middleware: Software that acts as an intermediary between different systems or applications, facilitating their interaction and communication.
  3. Information processed by the system
  4. User: people involved in using the system

## Inside the CPU



### • Apple A5

Figure 2: Apple A5 CPU

- The CPU has many core configurations. Here are some:
- Single core: One processing unit.
- Dual core: Two processing units.
- Quad core: Four processing units.
- Octa core: Eight processing units.
- Multi-core: General term for processors containing two or more independent cores.

Furthermore, there is deca-core (for 10 core)..., basically the more the cores the better the multitasking and parallel processing.

- Increased capacity and performance
- Reduced cost



| Year | Technology                 | Relative performance/cost |
|------|----------------------------|---------------------------|
| 1951 | Vacuum tube                | 1                         |
| 1965 | Transistor                 | 35                        |
| 1975 | Integrated circuit (IC)    | 900                       |
| 1995 | Very large scale IC (VLSI) | 2,400,000                 |
| 2013 | Ultra large scale IC       | 250,000,000,000           |

## §1.5 Technologies for Building Processors and Memory

Figure 3: Tech trends in recent decades

## Moore's Law

Gordon Moore (co-founder of Intel) observed in 1965 that the number of transistors on a microchip doubles approximately every two years, while the cost of computers is halved. This trend has driven the rapid advancement of computing power for decades, though it is currently slowing down as we approach the physical limits of silicon manufacturing.

Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World  
in Data

## Transistor count

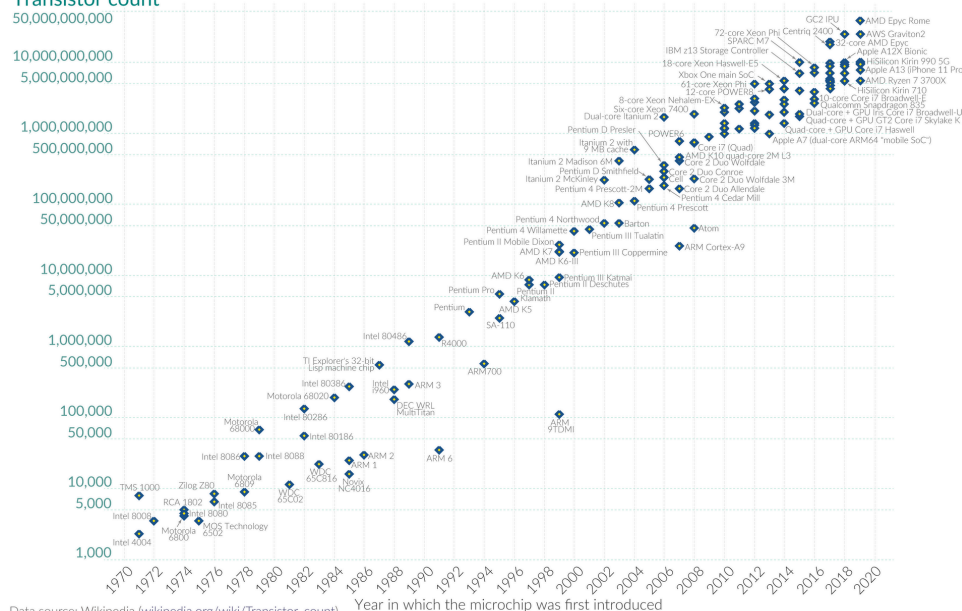


Figure 4: As of 2015: progress has slowed, doubling every 2.5 years

## Components

- Application software: is written in some higher level language.
- System software:
  - ▶ Compiler: translates high level code to machine code, can be a second step.

- OS: virtualization, parallelism, and concurrency
- Hardware: processor, memory, I/O controllers...

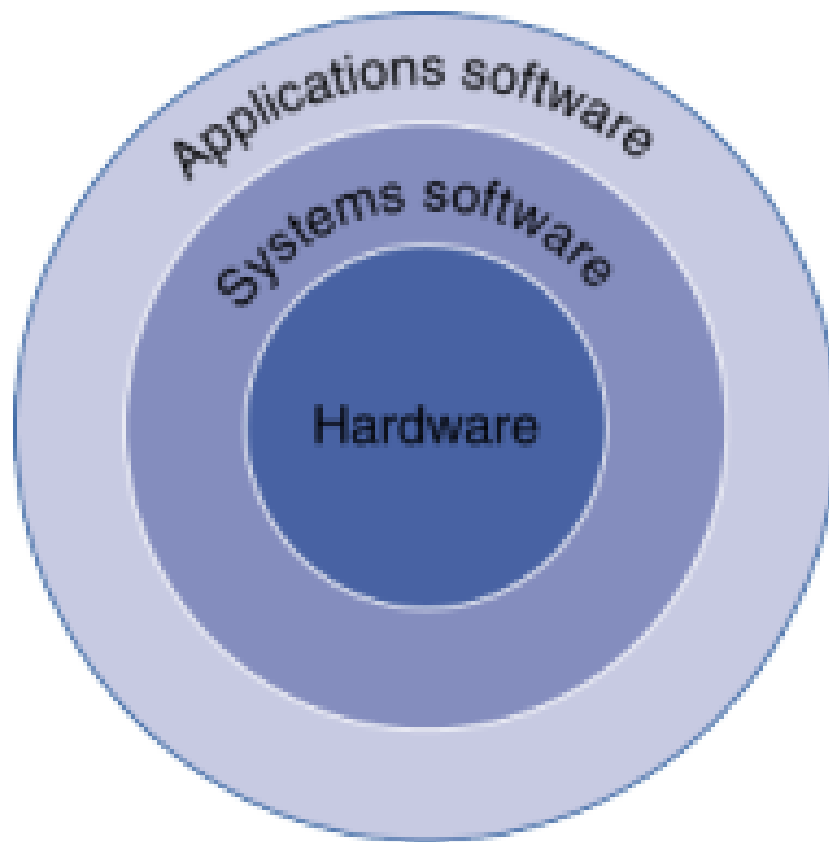


Figure 5: Components

## Instruction Sets

- Is defined as a collection of operations or commands that a computer's CPU can execute.
- The ISA defines the interface between the hardware and the software; meaning that different computers that different ISAs, but they are common in many ways.
- Speaking of ISAs, there are usually two specific types:
  1. CISC (Complex Instruction Set Computer): Provides many complex instructions that perform multiple operations in a single line (Example: x86).
  2. RISC (Reduced Instruction Set Computer): Provides simpler instructions that each take one clock cycle to complete (Examples: ARM, RISC-V, MIPS).
- Examples of ISAs:
  1. MIPS (Microprocessor without Interlocked Pipelined Stages): A RISC ISA often used in academia and embedded systems due to its simplicity.
  2. x86 (Intel 8086 family)
  3. ARM (Advanced RISC Machine)
  4. RISC-V (Reduced Instruction Set Computer V)
- ISAs consists of three basic instructions:
  1. Arithmetic/bitwise logic
  2. Data transfers between registers/memory

- 3. Control flow, jumping, functions
- Level of Program Code: High-Level then to Assembly then to Hardware Rep (bits)

Example:

```
f = (g + h) - (i + j);
add t0, g, h
add t1, i, j
sub f, t0, t1
```

## Components of The CPU

The CPU consists of three main components:

- CU (Control unit)
- ALU
- Registers
- Cache
- Buses
- Clock

## Clock Rate and Registers

- The CPU clock coordinates when operations happen inside the processor.
- Clock rate is usually measured in hertz, such as MHz or GHz.
- A higher clock rate means more clock cycles per second, but performance also depends on instruction design, pipelining, cache, memory speed, and number of cores.
- A **register** is a very small, very fast storage location inside the CPU.
- CPU registers are used for:
  1. operands for arithmetic and logic.
  2. addresses for memory access.
  3. temporary results.
  4. control information such as the program counter.
- Important register examples:
  1. **Program counter**: address of the next instruction.
  2. **Instruction register**: currently fetched instruction.
  3. **General-purpose registers**: hold values used by programs.
  4. **Stack pointer**: tracks the current top of the stack.

## Logic Gates

Computer are built of tiny switches (that can turn on (1) and off (0)) called transistors, by combining a bunch of transistors you can get **logic gates**.

Examples:

- AND : Output is 1 only if both inputs are 1.
- OR : Output is 1 if either both inputs are 1.
- NOT : Flips the input (1 → 0, 0 → 1)
- NAND & NOR : All in one circuits, meaning they are functionally complete.

Combination Circuits are circuits that only look at the current input (Ex: Logic Gates). They don't "remember" anything.

Sequential Circuits, on the other hand, looks at the current input and what happened in the past. To do this, they use Flip-Flops, which act as a 1 bit cell.

To watch: <https://www.youtube.com/watch?v=Hi7rK0hZnfc>

### **MUX, DeMUX, and Flip-Flops**

- A **multiplexer** or MUX selects one of many inputs and forwards it to one output.
- A MUX is controlled by select lines.
- Example: a 4-to-1 MUX has four inputs, one output, and select bits that choose which input is passed through.
- A **demultiplexer** or DeMUX does the reverse: it takes one input and routes it to one of many outputs.
- MUX and DeMUX circuits are useful in CPUs because hardware often needs to choose between several possible data sources or destinations.
- A **flip-flop** stores one bit of state.
- Flip-flops are the building blocks for registers, counters, and memory cells.
- Difference:
  1. combinational logic depends only on current inputs.
  2. sequential logic depends on current inputs and stored state.

### **Memory**

- SRAM (Static RAM): Used for Cache (close to the CPU). It is very fast, expensive, and uses multiple transistors per bit, not just one. It does not need to be refreshed.
- DRAM (Dynamic RAM): Used in Main Memory. It is slower but cheaper, denser, and uses a capacitor to store charges. Since these capacitor can leak, they need to be refreshed.
- Flash & Disk: Used for long term storage.

## Practice

1. Given the MIPS instruction give a run down on the 5 CPU stages to make this instruction happen.

`add $s0, $s1, $s2`

| Stage | Action for <code>add \$s0, \$s1, \$s2</code>                             |
|-------|--------------------------------------------------------------------------|
| IF    | Fetch the instruction from memory and increment the Program Counter.     |
| ID    | Decode instruction; read values of \$s1 and \$s2 from the Register File. |
| EX    | ALU calculates the sum of the values from \$s1 and \$s2.                 |
| MEM   | No memory access needed; the sum simply passes through this stage.       |
| WB    | Write the sum result back into the register \$s0.                        |

2. Convert the C code into MIPS ASM.

```
if ( i != j ) f = g + h;  
else f = g - h;
```

Solution:

```
# Compare i and j  
    bne $s3, $s4, L1    # if (i != j) goto L1  
  
    # Else block: f = g - h  
    sub $s0, $s1, $s2    # f = g - h  
    j L2                # jump to end (skip L1)
```

```
L1: # If block: f = g + h  
    add $s0, $s1, $s2    # f = g + h
```

```
L2: # End
```

3. Fill in the cells

|              | 1. Arithmetic | 2. Load | 3. Store | 4. Branch |
|--------------|---------------|---------|----------|-----------|
| Registers    | ✓             | ✓       | ✓        | ✓         |
| ALU          | ✓             | ✓       | ✓        | ✓         |
| Data Memory  | x             | ✓       | ✓        | x         |
| Control Unit | ✓             | ✓       | ✓        | ✓         |
| PC           | ✓             | ✓       | ✓        | ✓         |